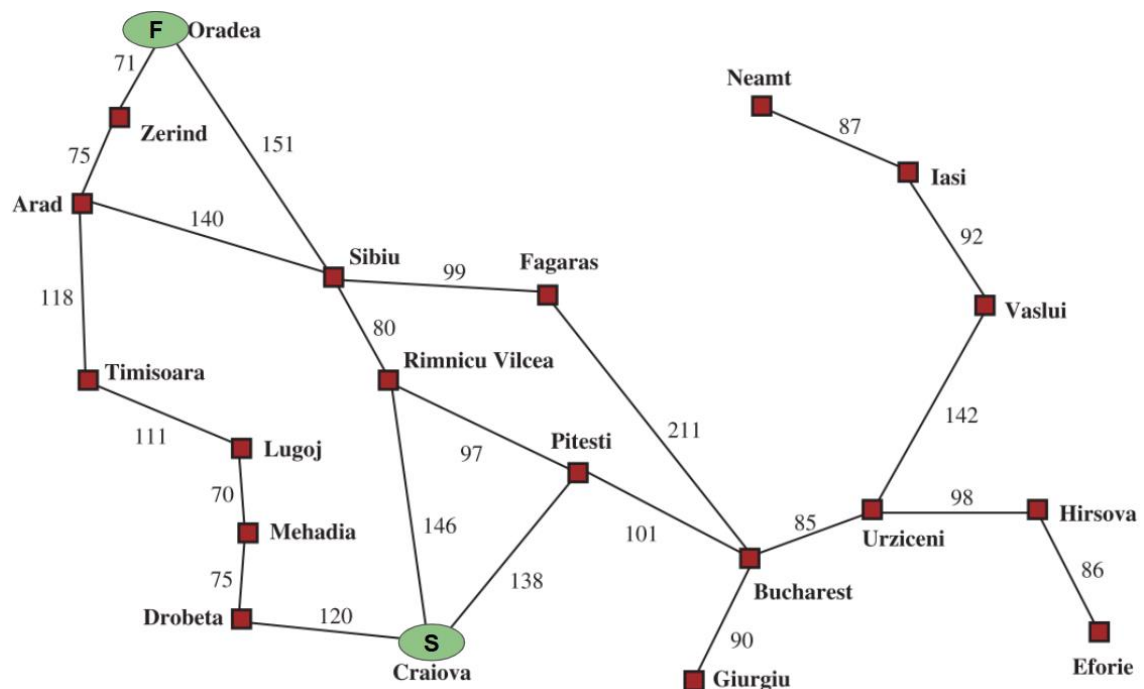


Búsqueda Informada – Reporte del Ejercicio 01

Elección de ciudades

Para este ejercicio elegí Craiova-Oradea. La razón principal es que el nodo de origen tiene 3 distintos nodos como primera parada, lo que creo que será útil para un análisis más completo.



S – start ; F – finish.

Explorar el mapa y confirmar la existencia de las ciudades

Craiova:

```
(venv) PS C:\Users\Osito\Desktop\inteligencia-artificial\Búsqueda informada\project> python 01_romania_map.py --from-city Craiova
Romania road map (AIMA Figure 3.2)
Cities: 20 Roads: 23

Craiova: Drobeta 120 km, Pitesti 138 km, Rimnicu Vilcea 146 km
```

Oradea:

```
(venv) PS C:\Users\Osito\Desktop\inteligencia-artificial\Búsqueda informada\project> python 01_romania_map.py --from-city Oradea
Romania road map (AIMA Figure 3.2)
Cities: 20 Roads: 23

Oradea: Sibiu 151 km, Zerind 71 km
```

Imprimir $h(x)$ hacia mi destino

Mi destino es Oradea:

```
(venv) PS C:\Users\Osito\Desktop\inteligencia-artificial\Búsqueda informada\project> python 02_heuristics.py --from-city Craiova --to Oradea
Heuristic: Euclidean distance to Oradea (map coordinates)

h(n) city
0 Oradea <- goal
46 Zerind
89 Arad
137 Sibiu
165 Timisoara
191 Rimnicu Vilcea
195 Lugoj
213 Fagaras
235 Mehadia
274 Drobeta
277 Neamt
277 Pitesti
308 Craiova <- start
```

Las ciudades vecinas y sus funciones eurísticas son:

- Drobeta, $h(n) = 274$
- Rimnicu Vilcea, $h(n) = 191$
- Pitesti, $h(n) = 277$

Nota: Greedy Best First Search debería elegir Rimnicu Vilcea como su primera opción, dado que su $h(n)$ es la menor de las 3.

Ejecutar ambos algoritmos

Greedy BFS:

```
(venv) PS C:\Users\Osito\Desktop\inteligencia-artificial\Búsqueda informada\project> python 03_greedy_best_first_search.py --from-city Craiova --to Oradea
Algorithm: Greedy best-first search
Problem: Craiova → Oradea
Heuristic: Euclidean distance to Oradea (map coordinates)
Status: success
Path: Craiova → Rimnicu Vilcea → Sibiu → Oradea
Depth: 3 roads
Cost: 377 km

city      g      h      f
Craiova   0      308    308
Rimnicu Vilcea 146    191    337
Sibiu     226    137    363
Oradea    377     0     377

Expanded: 3 nodes
Generated: 11 nodes
Frontier: max size 5
```

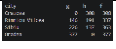
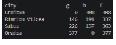
A* Search:

```
(venv) PS C:\Users\Osito\Desktop\inteligencia-artificial\Búsqueda informada\project> python 04_a_star_search.py --from-city Craiova --to Oradea
Algorithm: A* search
Problem: Craiova → Oradea
Heuristic: Euclidean distance to Oradea (map coordinates)
Status: success
Path: Craiova → Rimnicu Vilcea → Sibiu → Oradea
Depth: 3 roads
Cost: 377 km

city      g      h      f
Craiova   0      308    308
Rimnicu Vilcea 146    191    337
Sibiu     226    137    363
Oradea    377     0     377

Expanded: 3 nodes
Generated: 11 nodes
Frontier: max size 5
```

Tabla comparativa:

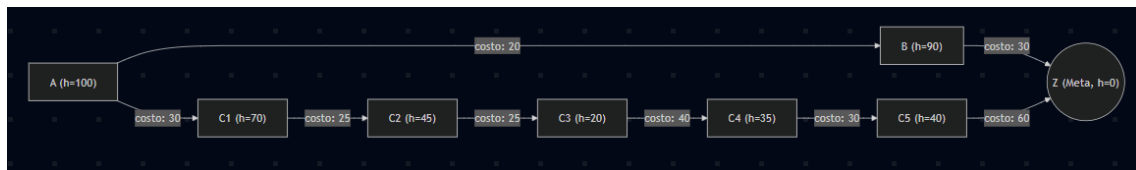
Nombre	Status	Path	Depth	Cost	Expanded	Generated	Tabla
Greedy BFS	Success	Craiova → Rimnicu Vilcea → Sibiu → Oradea	3	377	3	11	
A* Search	Success	Craiova → Rimnicu Vilcea → Sibiu → Oradea	3	377	3	11	

Reporte de resultados.

Tanto A* Search como Greedy Best First Search encontraron el camino con menos kilómetros. Originalmente, pensé que habría una diferencia evidente en los caminos tomados entre ambos algoritmos.

¿Por qué podría Greedy devolver un camino más caro a pesar de que h sea admisible? Principalmente porque h es el único referente/parámetro que toma en cuenta para la elección del siguiente nodo, ignorando el costo acumulado de la ruta $g(n)$.

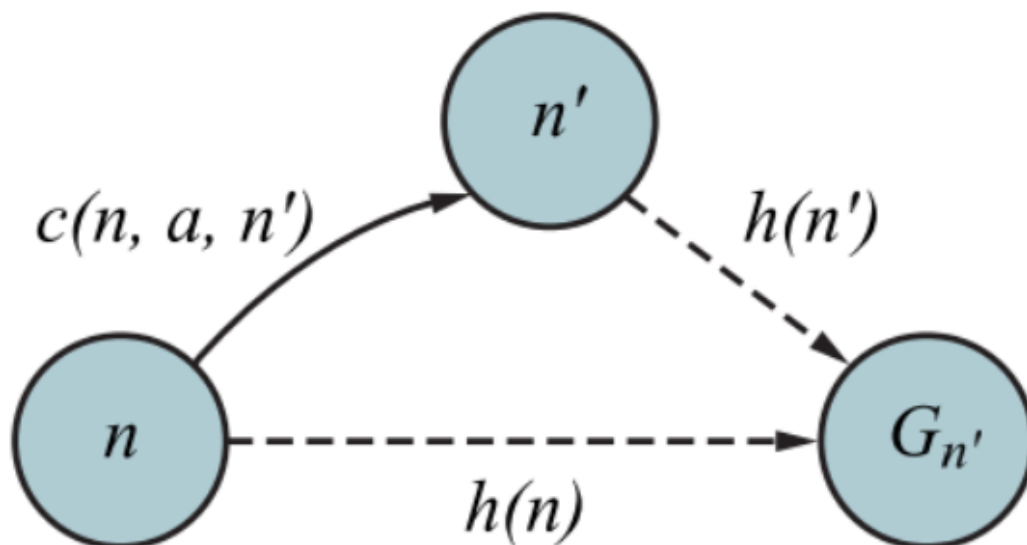
Ejemplo 1 – Exagerar el escenario



En el camino de A, ¿ f tiende a **no disminuir** a lo largo de la ruta?
Relaciónalo con que h sea consistente (en particular si el destino es Bucharest y se usa la tabla AIMA).*

Sí, la función $f(n)$ tiende a no disminuir, y podemos apoyarnos de la definición de consistencia de AIMA (página 106);

$$h(n) \leq c(n, a, n') + h(n')$$

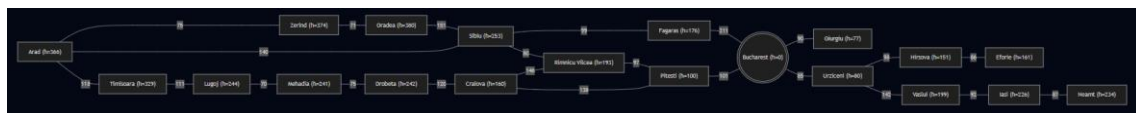


Como se ve en la imagen, cuando la heurística es consistente, $h(n)$ siempre será menor que la suma de costo desde n hasta n' + la heurística de n' . Como se

describe en el libro, es una propiedad de inequidad del triángulo: la suma de dos lados (en este caso, los lados que no son $h(n)$), SIEMPRE será mayor que $h(n)$. De esta forma, podemos ver que no hay manera en que, conforme A^* avance, su valor de evaluación $f(n)$ sea cada vez menor.

Reto Opcional

Para no tener problemas con la visualización del grafo, usé una herramienta para visualizarlo mejor:

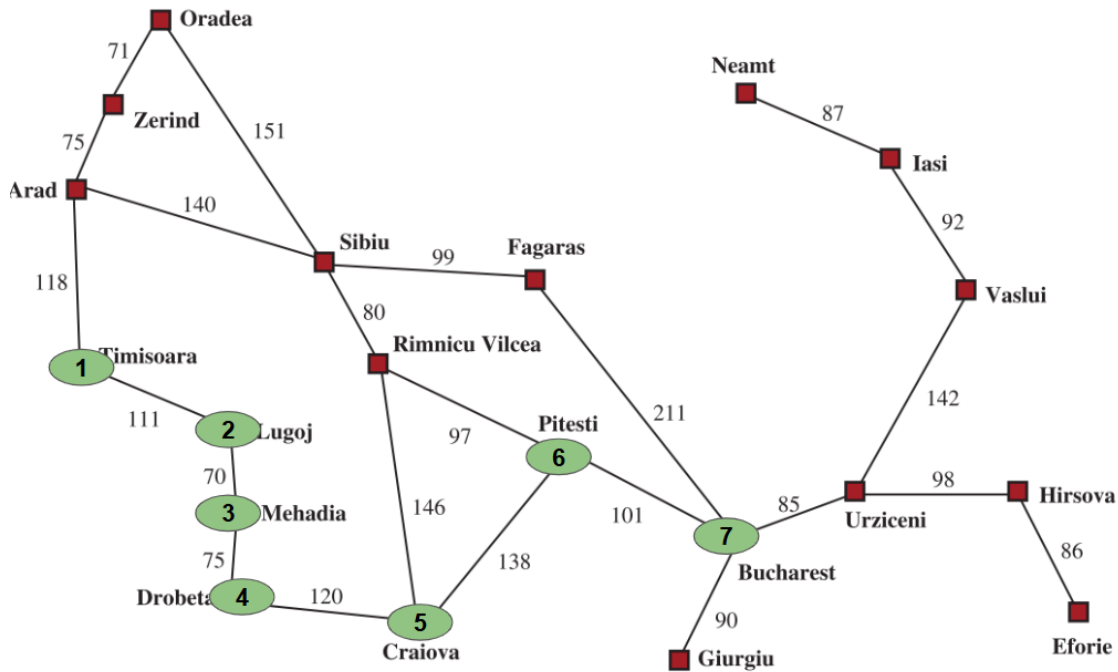


La pareja que permite un buen análisis es Bucharest-Timisoara.

Al correr Greedy FS, puede verse que la hipótesis es correcta. Al sólo tomar en cuenta $h(n)$, terminó recorriendo más kilómetros al final:

```
(venv) PS C:\Users\Osito\Desktop\inteligencia-artificial\Búsqueda informada\project> python 03_greedy_best_first_search.py --from-city Timisoara --to Bucharest
Algorithm: Greedy best-first search
Problem: Timisoara -> Bucharest
Heuristic: straight-line distance to Bucharest (AIMA table)
Status: success
Path: Timisoara -> Lugoj -> Mehadia -> Drobeta -> Craiova -> Pitesti -> Bucharest
Depth: 6 roads
Cost: 615 km
```

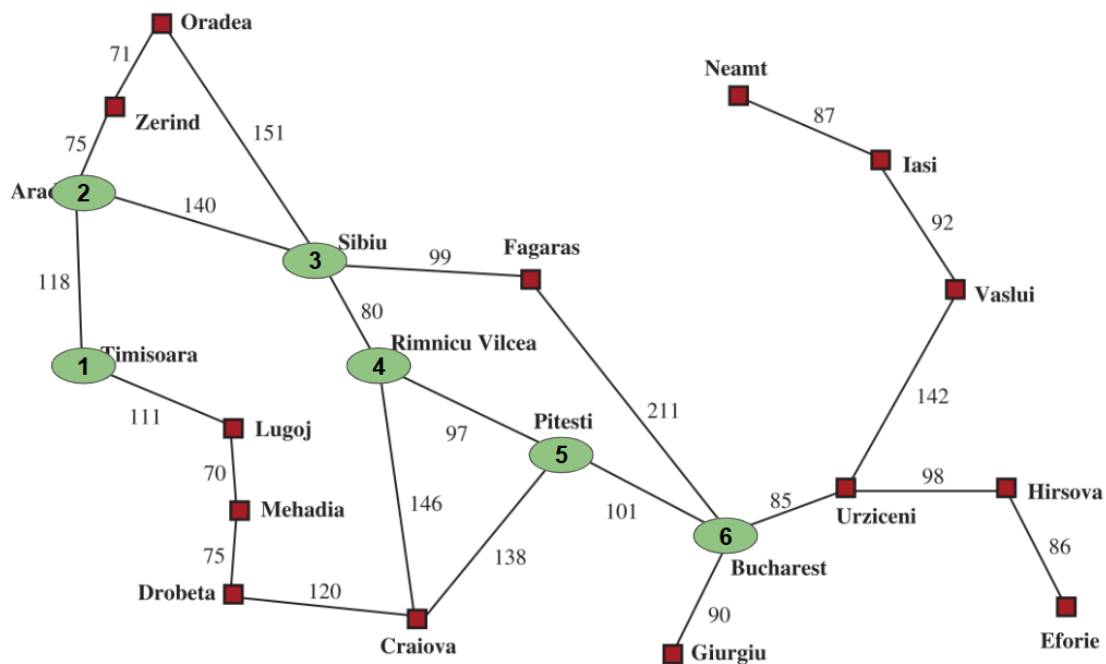
Este fue el camino tomado:



Al correr A*, puede observarse que el costo en kilómetros fue menor:

```
(venv) PS C:\Users\Osito\Desktop\inteligencia-artificial\Búsqueda informada\project> python 04_a_star_search.py --from-city Timisoara --to Bucharest
Algorithm: A* search
Problem: Timisoara -> Bucharest
Heuristic: straight-line distance to Bucharest (AIMA table)
Status: success
Path: Timisoara -> Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Depth: 5 roads
Cost: 536 km
```

Este fue el camino tomado por A*:



- Correr la pareja con UCS

```
(venv) PS C:\Users\osito\Desktop\inteligencia-artificial\Búsqueda no informada\project> python 03_uniform_cost_search.py --from-city Timisoara --to Bucharest
Algorithm: Uniform-cost search
Problem: Timisoara -> Bucharest
Status: success
Path: Timisoara -> Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Depth: 5 roads
Cost: 536 km
Expanded: 12 nodes
Generated: 31 nodes
Frontier: max size 4
```

Como puede verse en la imagen, el costo fue de 536 km, que efectivamente coincide con el costo generado por A*.

Cambiando los destinos – Greedy

Para la siguiente pareja, elegí Timisoara a Sibiu

Para Greedy, el costo fue 258 kilómetros:

```
(venv) PS C:\Users\osito\Desktop\inteligencia-artificial\Búsqueda informada\project> python 03_greedy_best_first_search.py --from-city Timisoara --to Sibiu
Algorithm: Greedy best-first search
Problem: Timisoara -> Sibiu
Heuristic: Euclidean distance to Sibiu (map coordinates)
Status: success
Path: Timisoara -> Arad -> Sibiu
Depth: 2 roads
Cost: 258 km
```

Para A*, también:

```
(venv) PS C:\Users\osito\Desktop\inteligencia-artificial\Búsqueda informada\project> python 04_a_star_search.py --from-city Timisoara --to Sibiu
Algorithm: A* search
Problem: Timisoara -> Sibiu
Heuristic: Euclidean distance to Sibiu (map coordinates)
Status: success
Path: Timisoara -> Arad -> Sibiu
Depth: 2 roads
Cost: 258 km
```

En este caso, Greedy, tomando sólo la heurística como “brújula”, pudo encontrar el camino menos costoso en km, coincidiendo con la ruta óptima propuesta por A*. Este resultado fue causado por la geometría del grafo: el camino dado por la heurística coincidió con las carreteras con menor distancia en kilómetros.

